Emotion text classification. NLP Course Project

Aleksey Heleskul

May 2025

Abstract

This paper presents a natural language processing (NLP) approach to classifying emotions in user-generated text from an online platform such as Twitter (X). Using a labeled data set of user-generated messages, we trained and evaluated several machine learning models to identify the emotional states expressed in the text. The final model classifies emotions such as joy, anger, sadness, love, and fear with a macro F1-score of 88% and an accuracy of 92%. Our results demonstrate the potential of NLP-related methods to correctly identify emotional patterns that can support mental health monitoring and content moderation. The source code for the project is available: `https://github.com/oopscompiled/nlp-project`.

1 Introduction

As the online community evolves and provides powerful tools for self-expression – such as X (Twitter), Reddit, or Discord comments – the analysis of emotional content in user texts is becoming increasingly important. This is due, in particular, to the need to identify and prevent potential threats associated with expressions of anger, depression, or other strong emotions, such as sadness, which is often found, for example, in Facebook messages. Natural language processing (NLP) tools become especially important when people interact with another person indirectly, such as a chatbot or voice assistant. Such technologies are increasingly used in customer support, banking, and medical consultations. But what if the person on the other end of the line is stressed or even in crisis? If a system can recognize emotions such as fear, anxiety, or despair, it can do more than just 'answer according to a script' - it can, for example, transfer the call to a live specialist or activate a more sensitive communication script. A good emotion classification model in such cases is not just an algorithm, but a truly important support tool that can make a difference in someone's life.

1.1 Team

This project was fully completed individually by Aleksey Heleskul.

2 Related Work

Emotion analysis (EA) is a subfield of Natural Language Processing (NLP) that focuses on identifying and interpreting emotions expressed in text. Although closely related to sentiment analysis, which typically distinguishes between positive, negative, and neutral sentiments. One of the primary goals of EA is to capture a wider and more nuanced range of emotional states such as joy, sadness, anger, fear, love and surprise. Throughout the years, many researchers across different fields have explored various approaches to emotion classification [Plaza-del Arco et al., 2024], starting from lexicon-based approaches, rule-based systems and up to machine learning (ML) and deep learning models (DL), particularly with the birth of large-scale annotated datasets and transformer-based architectures like BERT [Devlin et al., 2019] and next generation of it. In our study, we also refer to the work of Kristína Machová et al. [Machová et al., 2023], titled *Detection of Emotion by Text Analysis Using Machine Learning*, which was served as a baseline for selecting evaluation metrics and general methodological direction. The authors proposed a machine learning approach to emotion classification and reported an accuracy of approximately 90%.

However, after a closer inspection, it appears that this result may have been due to a methodological error. Specifically, the model was evaluated using the same dataset for both training and testing, which led to data leakage and significantly overestimated performance. In particular, tokenization and feature extraction were applied to the full dataset prior to the train-test split, which in turn allowed information that should have been hidden from the test set influenced the neural network training process.

However, our own re-implementation with proper data separation and cross-validation has achieved accuracy close to 67% with a macro f1-score of 50% , indicating that the initial estimate of 90% was most likely overestimated due to an incorrect evaluation protocol.

This case highlights the importance of thorough experimental design and underlines the need for strict separation between training and evaluation data in a machine learning related tasks — especially in emotion recognition, where overfitting is quite a frequent challenge.

3 Model Description

Our emotion classification pipeline is based on a hybrid architecture that combines pretrained transformer-based embeddings with custom neural classifiers. We experiment with three deep learning models: a bidirectional LSTM (**MyLSTM**), a multi-layer GRU (**MyGRU**), and a hybrid CNN-LSTM with attention (**HybridNN**). All models operate on sentence embeddings obtained from the DeBERTa-base transformer.

3.1 Pretrained Embeddings (DeBERTa)

As a feature extractor, we use the pretrained DeBERTa-base model (`microsoft/deberta-base`), which produces contextualized embeddings of dimension $d = 768$ for each input token. Let $X = [x_1, x_2, \ldots, x_T]$ be a tokenized input of length T. The model outputs embeddings $E = [e_1, e_2, \ldots, e_T]$, where $e_i \in \mathbb{R}^{768}$.

3.2 Projection and Dropout

In order to reduce computational cost and enable more efficient training of downstream classifiers, we apply a linear projection:

$$e'_i = \text{LayerNorm}(W_{\text{proj}} e_i + b), \quad W_{\text{proj}} \in \mathbb{R}^{d' \times 768}$$

where d' is typically a value of 128 or 256 (in our case 128), depending on the model. On the other hand, dropout is applied to the projected embeddings before passing them into the recurrent layers.

3.3 MyLSTM

The MyLSTM model consists of a bidirectional LSTM layer with hidden size $h = 128$ and two layers. After processing the sequence stage, we extract the final hidden states from both directions:

$$h_{\text{final}} = \text{Dropout}([h_{\rightarrow}^{(L)}, h_{\leftarrow}^{(L)}])$$

This vector is passed through a GELU activation and a fully connected classifier layer. In our architecture, we employed the GELU (Gaussian Error Linear Unit) activation function instead of more traditional choices of activation functions such as ReLU or tanh. GELU is defined mathematically as $\text{x} \cdot \Phi(\text{x})$, where $\Phi(\text{x})$ is the cumulative distribution function of the standard normal distribution.

GELU offers several potentially beneficial properties for our task. Its smooth, differentiable nature provides continuous gradients throughout the input range, unlike ReLU which has zero gradients for negative inputs. The function's slight non-monotonicity (small negative values for negative inputs) may help preserve information that could be relevant for nuanced classification tasks like emotion recognition. Additionally, GELU's probabilistic foundation aligns well with the stochastic nature of neural network training.

On the other hand, while GELU doesn't directly solve the vanishing gradient problem, its smooth gradient characteristics can assist in more stable training in deep networks compared to hard-threshold functions like ReLU, as illustrated in Figure 1. GELU shows here a more continuous activation landscape, which may be particularly suitable for this task as it requires processing complex linguistic patterns where precise contextual information is highly important.

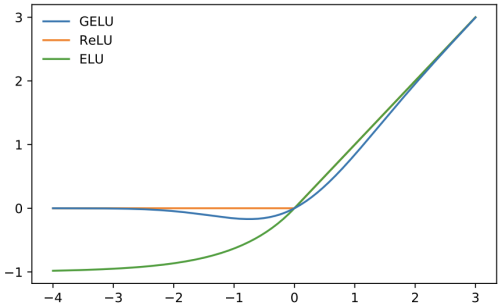

Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

Figure 1: Comparison of GELU activation with other activation functions

3.4 MyGRU

MyGRU is structurally similar to MyLSTM but uses Gated Recurrent Units (GRU) instead. It has two stacked GRU layers with a hidden size of 128. The output representation is again obtained from the final hidden states:

$$h_{\text{final}} = \text{Concat}(h_{\text{last}}^{\rightarrow}, h_{\text{last}}^{\leftarrow})$$

The use of GRU leads to faster training due to a simpler architecture and a fewer trainable parameters with a comparable performance comparing to other deep learning models described here.

3.5 HybridNN (CNN-LSTM with Attention)

This hybrid architecture builds upon the foundational work of [Zhou et al., 2015], who proposed a hybrid model in order to extract a sequence of higher-level phrase representations C-LSTM (Convolutional-LSTM) approach in a paper "A C-LSTM Neural Network for Text Classification". We adapt their methodology to grasp both convolutional feature extraction and sequential modeling for our emotion from text classification task.

The HybridNN architecture first applies a 1D convolution over the sequence of embeddings, which produces C channels with kernel size of k:

$$C = \text{ReLU}(\text{Conv1D}(E))$$

After convolution and max pooling, the output is passed to a bidirectional LSTM. On top of the LSTM, we compute attention weights α_i over the sequence as a final "signal" layer:

$$\alpha_i = \frac{\exp(w^\top h_i)}{\sum_j \exp(w^\top h_j)}$$

The final sequence is represented as a weighted sum:

$$v = \sum_{i=1}^{T} \alpha_i h_i$$

where h_i are the LSTM outputs. Therefore, this attention-enhanced vector is then fed to the final classification layer.

3.6 Training Configuration

All models are trained with class-balanced cross-entropy loss:

$$\mathcal{L} = -\sum_{i=1}^{K} w_i y_i \log(\hat{y}_i)$$

where K is the number of classes, and w_i is the inverse frequency class weight, which all equal one by default.

It's important to note that `torch.nn.functional.cross_entropy` expects unnormalized logits as input, not probabilities. Given that, this function internally applies the softmax operation in order to convert logits to probabilities before computing the cross-entropy loss. This approach is numerically more stable than manually applying softmax followed by logarithm operations.

For optimization, we employ either Adam or AdamW optimizers with weight decay regularization. Learning rate scheduling is applied using either *ReduceLROnPlateau* (which reduces learning rate when validation loss plateau) or *CosineAnnealingWarmRestarts* (which uses cosine annealing with periodic restarts in order to assist in escaping local minima).

3.7 Data Augmentation

Given that the data was collected from messages on X (formerly Twitter) it is essential to properly clean them in a way of deleting stop words, some HTML, and other artifacts that may introduce some bias, overall affecting the model's performance.

Secondly, to address class imbalance, we perform controlled data augmentation for minority emotion classes (*sadness*, *fear*, *surprise*) using a semantic crossover technique and *Googletrans*, which is a Google Translate API wrapper for using back translation. Given two sentences $A = [a_1, ..., a_n]$ and $B = [b_1, ..., b_m]$, we generate new samples by swapping their second halves:

$$\text{new}_1 = a_{\lfloor n//2 \rfloor + 1:n} \| b_{\lfloor m//2 \rfloor + 1:m}, \quad \text{new}_2 = b_{1:\lfloor m//2 \rfloor} \| a_{\lfloor n//2 \rfloor + 1:n}$$

This strategy of data augmentation balances between speed and quality, adding some diversity while maintaining semantic coherence. In addition, we apply back translation (e.g., English → French → English) to further enrich the data with paraphrased variants. Simultaneously, these augmentations increase

the representation of minority classes and improve the model's generalization on underrepresented emotions.

3.8 Overview

An overview of the pipeline is shown in Figure 2.

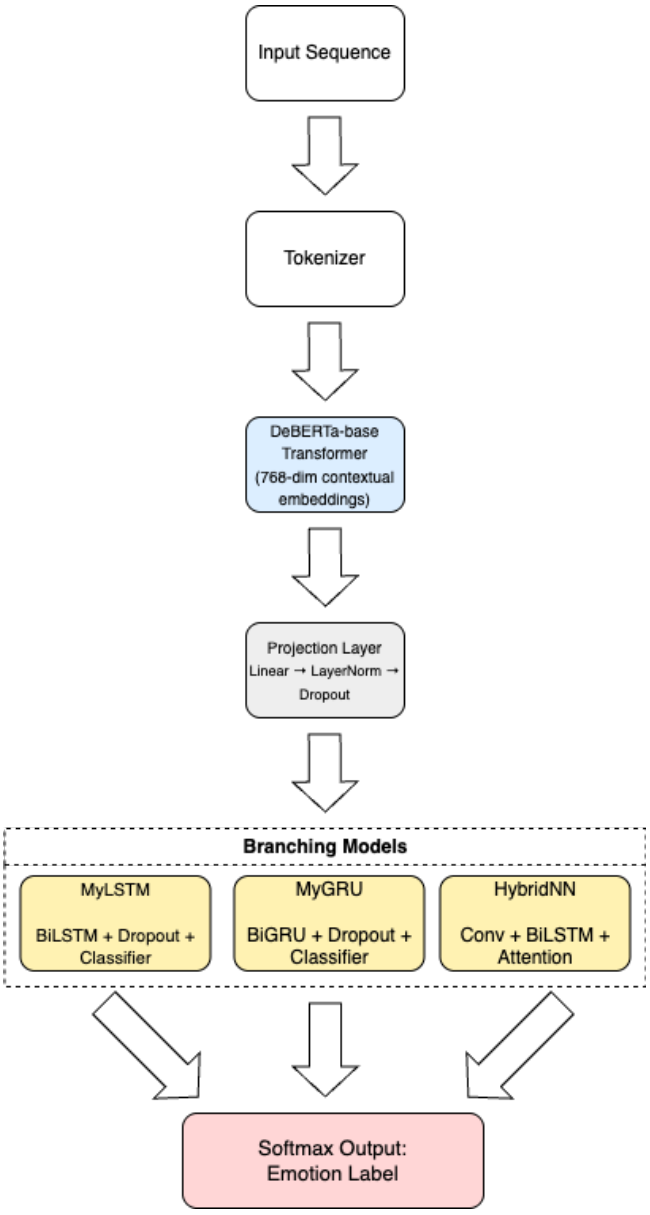

Figure 2: Pipeline: DeBERTa embeddings → projection → LSTM/GRU/CNN-LSTM → attention → classifier.

4 Dataset

For this study, we used the **Emotion Dataset for NLP** introduced by Praveen G. and available for research purposes via Kaggle[1]. The dataset comprises short English texts (commonly in a sentence or two), each labeled with one of six basic emotions: `joy`, `anger`, `sadness`, `fear`, `love`, and `surprise`. It was originally collected to support research in affective computing and emotion detection in text, especially in the field of ML techniques.

Each record in the dataset contains a `text` field with the input sentence and a corresponding `label` field denoting the associated emotion class. The dataset is well-suited for classification tasks and has been used in various NLP studies.

The data was split into training, validation, and test subsets with the ratio of 80:10:10 respectively, ensuring proper stratification across all the classes to ensure label balance across the splits.

Emotion Class	**Train**	**Validation**	**Test**
Joy	5362	704	695
Anger	2159	275	275
Sadness	4666	550	581
Fear	1937	212	224
Love	1304	178	159
Surprise	572	81	66
Total	16000	2000	2000

Table 1: Class-wise distribution of samples across dataset splits.

As shown in Table 1, the dataset is pretty imbalanced, particularly in classes like `love` and `surprise` which contain fewer records. To address this issue, we have applied a light data augmentation strategy to the diminished classes, which was further described in Section 3.

5 Experiments

5.1 Metrics

As it comes to the evaluation of the performance of our models on the emotion classification task, we have used the following metrics:

Accuracy — the proportion of correctly classified samples among all the samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

However, in this case we suggest not relying solely on this simple metric, as it does not provides any additional information how confident the model

[1] https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp

was in predicting, especially on an unbalanced data. In multiclass classification tasks, it is also important to consider the types of errors that model makes. For instance, confusion between certain classes may have different practical consequences. Therefore, some tools like confusion matrices and per-class metrics such as precision, recall, and F1-score together are crucially valuable to gain deeper insights into model performance beyond overall accuracy.

Macro F1-score — the unweighted mean of F1-scores across all the classes, which helps to address class imbalance. The precision, recall, and F1-score formulas are:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The Macro F1 is computed as the average of the F1-scores for all the classes.

Macro F1-score was chosen as the primary metric due to the facts that we have to treat the data with an imbalanced nature of the emotion classes as well as it balanced between recall and precision. Having such an evaluation mechanism at our disposal will make it much easier for us to decide whether the changes in the model are better or not.

5.2 Experiment Setup

We conducted experiments on the emotions dataset obtained from Kaggle. The dataset was split into training, validation, and test sets. Input sequences were truncated or padded to a maximum length of 64 tokens.

Tokenization was performed using the Microsoft DeBERTa-base tokenizer, known for its effectiveness in handling complex linguistic patterns.

The models used in our experiments include:

- **MyLSTM**: a bidirectional LSTM with two layers, embedding size of 128, and hidden dimension of 128 with a dropout applied to input and intermediate layers.
- **MyGRU**: a bidirectional GRU with two layers, embedding size of 128 and a hidden dimension of 128.
- **HybridNN**: a hybrid architecture combining convolutional layers, bidirectional LSTM, and an attention mechanism, enhanced with adaptive max pooling.

In order to avoid overfitting, a dropout with probability 0.3 was applied within the DeBERTa model by setting `bert.config.dropout = 0.3`. This regularization technique helps the model generalize better on unseen data.

Hyperparameters for training were as follows:

- Batch size: 64

- Optimizers: AdamW for HybridNN, Adam for MyLSTM and MyGRU.
- Learning rates: from 1×10^{-5} to 2×10^{-5} for the pretrained BERT layers, and from 2×10^{-4} to 2×10^{-3} for the classification layers.
- Learning rate scheduling strategies included ReduceLROnPlateau and CosineAnnealingWarmRestarts.
- Number of epochs: 10–30, with early stopping based on validation performance.

6 Results

Table 2 summarizes the test set results.

Table 2: Performance of all models on the test set.

Model	**Accuracy**	**Macro F1**	**Weighted F1**
MyLSTM	0.86	0.80	0.86
MyGRU	0.92	0.88	0.84
HybridNN	0.92	0.88	0.92
Logistic Regression (TF-IDF)	0.86	0.83	0.87
Random Forest (BoW)	0.87	0.85	0.87

From Table 2, we observe that all the models achieve moderately high performance, indicating that the dataset is well-suited for emotion classification. What's interesting is that even simple models such as Logistic Regression and Random Forest perform on par with more complex neural architectures in terms of accuracy and weighted F1-score. This may suggests us that traditional vectorization techniques like TF-IDF and BoW can still be highly effective.

However, when we take a look at the macro F1-score — which better captures the performance across all classes, including less frequent ones — the HybridNN model shows a slight advantage. This indicates that deep learning models with attention mechanism and sequence modeling may offer better generalization across diverse emotional categories.

Custome LSTM architecture also shows solid performance but slightly underperform compared to the convolutional hybrid and classical baselines, possibly due to fewer layers or lack of attention mechanisms.

7 Conclusion

In this work, we have explored the task of emotion classification in text using both traditional machine learning approaches and modern deep neural network architectures. We experimented with a dataset containing labeled emotional expressions and compared the performance of several models.

First, we applied two baseline techniques: Logistic Regression utilizing TF-IDF features and Random Forest using Bag-of-Words. These traditional methods delivered unexpectedly robust results, reaching accuracy levels of up to 92% and a macro F1-score of 88%. This demonstrates that with well-designed features, straightforward models can remain very competitive.

Futhermore, we created and assessed three neural models: a BiLSTM classifier, a BiGRU classifier, and a hybrid structure that integrates CNN, LSTM, and attention mechanisms. The HybridNN model stood best among the others, yielding the best overall results, slightly exceeding the baselines in macro F1-score while also sustaining high accuracy.

We also analyzed the models concerning their inference time and computational efficiency. Although the neural networks needed GPU acceleration for training, their inference durations stayed reasonable for deployment. On the other hand, baseline models provided quicker inference times and were simpler to train on conventional CPU hardware, making them appropriate for applications with limited resources.

References

[Devlin et al., 2019] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

[Machová et al., 2023] Machová, K., Szabóová, M., Paralič, J., and Mičko, J. (2023). Detection of emotion by text analysis using machine learning. *Frontiers in Psychology*, 14.

[Plaza-del Arco et al., 2024] Plaza-del Arco, F. M., Martín-Valdivia, M.-T., Ureña-López, L. A., and Montejo-Ráez, A. (2024). A survey on emotion classification approaches: From rule-based to deep learning. *arXiv preprint arXiv:2403.01222*.

[Zhou et al., 2015] Zhou, C., Sun, C., Liu, Z., and Lau, F. (2015). A c-lstm neural network for text classification. *arXiv preprint arXiv:1511.08630*.